

C#/.NET Koleksiyon Türlerinin Karşılaştırmalı Analizi: IEnumerable, IQueryable, ICollection, IList ve List

Berat Soylu

Bilgisayar Mühendisliği Bölümü, Trakya Üniversitesi, Edirne

Özet — C#/.NET platformunda bir koleksiyonu temsil eden türlerin çoğu ortak bir arayüz hiyerarşisinden türer ve aralarındaki ayrımlar yeterince anlaşılmadığında, işlevsel olarak çalışan ancak verimsiz yazılımlar ortaya çıkabilir. Bu çalışmada *IEnumerable<T>*, *IQueryable<T>*, *ICollection<T>*, *IList<T>* ve somut *List<T>* sınıfı; sundukları yetenekler, çalışma yerleri, değiştirilebilirlik ve erişim biçimleri açısından karşılaştırmalı olarak incelenmektedir. Özellikle Entity Framework Core bağlamında *IEnumerable* ile *IQueryable* ayrımının sorgu performansı üzerindeki etkisi ele alınmakta ve tür seçimine ilişkin uygulama önerileri sunulmaktadır.

Anahtar Kelimeler — C#, .NET, koleksiyon türleri, LINQ, Entity Framework Core, IQueryable, IEnumerable, gecikmeli çalışma.

1. Giriş

Bir yöntem tasarlanırken geriye döndürülecek koleksiyonun türüne karar vermek, ilk bakışta önemsiz görünen ancak sonuçları bakımından belirleyici bir tercihtir. *List<T>*, *IEnumerable<T>*, *ICollection<T>* ve *IQueryable<T>* gibi türlerin tümü söz dizimsel olarak geçerlidir ve derlenir. Bununla birlikte, bir yazılımın *çalışması* ile *doğru tasarlanmış olması* arasında belirgin bir fark bulunmaktadır. Bu fark, özellikle nesne-ilişkisel eşleme (ORM) araçlarının, örneğin Entity Framework Core'un, kullanıldığı senaryolarda kendini göstermektedir; yanlış bir tür seçimi, veritabanına gereksiz yük bindiren bir sorgu ile yalnızca gerekli verinin çekildiği verimli bir sorgu arasındaki farkı doğurabilmektedir.

Bu çalışmada söz konusu türler tek tek ele alınmakta, aralarındaki kalıtım ilişkisi ortaya konmakta ve her bir türün hangi durumlarda tercih edilmesi gerektiği tartışılmaktadır. İnceleme, kavramsal açıklamaların yanı sıra kısa kod örnekleriyle desteklenmektedir.

2. Ortak Kalıtım Hiyerarşisi

İncelenen türlerin büyük bölümü birbirinden türeyen **arayüzlerdir (interface)**. Buna karşılık *List<T>*, bu arayüzleri uygulayan somut bir **sınıftır (class)**. Türler arasındaki kalıtım ilişkisi temel olarak aşağıdaki gibidir:

```

IEnumerable<T>      → Dolaşılabilir (foreach)
├── IQueryable<T>   → Sorgu veritabanına çevrilir
├── ICollection<T>  → Sayılabilir, eleman eklenip çıkarılabilir
│   ├── IList<T>    → İndeksle erişilebilir
│   └── List<T>     → Somut sınıf; tüm işlemleri kapsar

```

Hiyerarşide alt seviyelere inildikçe türün sunduğu **yetenek artmakta**, buna karşılık **esneklik azalmaktadır**. `IEnumerable<T>` en az taahhütte bulunan tür iken (yalnızca dolaşma), `List<T>` tüm işlemleri kapsamaktadır. Bu ilke, ilerleyen bölümlerde değinilecek olan tür seçimi önerilerinin de temelini oluşturmaktadır: uygun tür çoğunlukla, gerçekten ihtiyaç duyulan en düşük yeteneği sunan türdür.

3. IEnumerable<T>

Hiyerarşinin kökünde yer alan `IEnumerable<T>`, tek bir taahhütte bulunur: üzerinde `foreach` döngüsüyle ileri yönde dolaşılabilir. Eleman sayısını doğrudan bilmez (sayım için baştan sona dolaşmak gerekir), eleman eklenip çıkarılamaz ve indeksle erişime izin vermez.

```

IEnumerable<int> sayilar = new List<int> { 1, 2, 3 };

foreach (var s in sayilar)
    Console.WriteLine(s);

```

Bu türün en belirleyici özelliği **gecikmeli çalışmadır (deferred execution)**. Bir LINQ sorgusu tanımlandığında sorgu hemen yürütülmez; sonuç fiilen kullanılabildiği kadar (örneğin `foreach` döngüsüne girildiğinde veya `ToList()` çağrıldığında) ertelenir.

```

IEnumerable<int> query = sayilar.Where(n => n > 1); // henüz yürütülmedi
foreach (var n in query)                               // filtreleme burada yürütülür
    Console.WriteLine(n);

```

`IEnumerable<T>` üzerinde tanımlanan LINQ işlemleri bellekte yürütülür; bu yaklaşım literatürde *LINQ to Objects* olarak adlandırılır ve verinin önceden belleğe alınmış olmasını gerektirir. Bu tür, bir koleksiyonun yalnızca okunup dolaşılacağı durumlarda, özellikle de yöntem parametrelerinde tercih edilir.

4. IQueryable<T>

Dış görünümü itibarıyla `IEnumerable<T>`'ye benzeyen ve ondan türeyen `IQueryable<T>`, iç işleyişi bakımından ondan ayrılır. İki tür arasındaki temel fark, sorgunun yürütüldüğü yerdir: `IEnumerable<T>` sorguyu bellekte çalıştırırken, `IQueryable<T>` sorguyu bir **ifade ağacı (expression tree)** biçiminde tutar, bunu SQL gibi bir sorgu diline çevirir ve veri kaynağında yürütür.

```
IQueryable<Kullanici> sorgu = _context.Kullaniciilar
    .Where(k => k.Yas > 18);
// Üretilen SQL: SELECT * FROM Kullaniciilar WHERE Yas > 18

var yetiskinler = sorgu.ToList(); // sorgu veritabanında burada yürütülür
```

Where, OrderBy ve Select gibi işlemler zincirlenebilir ve bunların tümü tek bir SQL sorgusunda birleşir. Bu sayede filtreleme işi veri kaynağı tarafından yapılır ve ağ üzerinden yalnızca gereksinim duyulan satırlar aktarılır. IQueryable<T> de gecikmeli çalışma ilkesine uyar; ToList(), First() veya Count() gibi bir işlem çağrılana kadar veri kaynağına erişilmez.

5. IEnumerable ve IQueryable Ayırımının Performansa Etkisi

Uygulamada en sık karşılaşılan performans sorunlarından biri, bu iki türün ayırt edilememesinden kaynaklanmaktadır. Aşağıdaki iki kod bloğu birbirine oldukça benzese de tamamen farklı biçimde yürütülür.

```
// Verimsiz: tüm tablo önce belleğe alınır, filtreleme sonrasında C# tarafında yapılır
IEnumerable<Kullanici> hepsi = _context.Kullaniciilar.AsEnumerable();
var yetiskinler = hepsi.Where(k => k.Yas > 18);
// SQL: SELECT * FROM Kullaniciilar (koşul içermez; tüm satırlar çekilir)
```

```
// Verimli: filtreleme veritabanında yürütülür, yalnızca gerekli satırlar aktarılır
IQueryable<Kullanici> yetiskinler = _context.Kullaniciilar.Where(k => k.Yas > 18);
// SQL: SELECT * FROM Kullaniciilar WHERE Yas > 18
```

Tasarım kuralı. Bir Entity Framework Core sorgusu AsEnumerable() ya da ToList() ile veya bir IEnumerable<T> değişkene atandığı andan itibaren, sonraki tüm LINQ işlemleri bellekte yürütülür. Bu nedenle filtreleme, sıralama ve izdüşüm (Select) işlemleri, sorgu henüz IQueryable aşamasındayken, yani veritabanına gönderilecek kısımda tanımlanmalı; sonucun belleğe alınması (materyalizasyon) ise mümkün olan en son adıma bırakılmalıdır.

6. ICollection<T>

ICollection<T>, IEnumerable<T>'nin üzerine somut yetenekler ekleyerek koleksiyonun bellekte hazır bulunmasını ve içeriğinin değiştirilebilmesini sağlar. Sunduğu başlıca üyeler şunlardır: Count özelliği koleksiyonu dolaşmadan eleman sayısını verir; Add, Remove ve Clear yöntemleri eleman ekleme, çıkarma ve temizleme işlemlerini gerçekleştirir; Contains ise belirli bir elemanın koleksiyonda bulunup bulunmadığını denetler.

```
ICollection<string> isimler = new List<string>();
isimler.Add("Ali");
isimler.Add("Veli");
isimler.Remove("Ali");
int adet = isimler.Count; // dolaşmadan → 1
bool varMi = isimler.Contains("Veli"); // → true
```

`ICollection<T>` sıralamayı ve indeksli erişimi garanti etmez; koleksiyonun sayılabilir ve değiştirilebilir olduğunu ifade eder, ancak belirli bir konumdaki elemana doğrudan erişim sağlamaz. Bu tür, çağırın kodun ekleme ve çıkarma yapmasına izin verilmek istenen, ancak indeksli erişime gerek duyulmayan durumlarda uygundur. Entity Framework Core'da, ilişkili kayıtları temsil eden gezinme özellikleri (navigation property) çoğunlukla `ICollection<T>` türünde tanımlanır; örneğin bir `Musteri` varlığının `ICollection<Siparis>` `Siparisler` özelliği bu kullanıma örnektir.

7. `IList<T>`

`IList<T>`, `ICollection<T>`'nin üzerine sıralama ve indeks kavramlarını ekler. Bu sayede elemanlara konumlarına göre erişilebilir ve koleksiyonun araya ekleme yapılabilir. Başlıca üyeleri arasında köşeli parantezle indeksli erişim sağlayan `this[int]`, belirli bir konuma ekleme yapan `Insert`, belirli konumdaki elemanı silen `RemoveAt` ve bir elemanın sırasını döndüren `IndexOf` yer alır.

```
IList<string> liste = new List<string> { "Ali", "Veli", "Ayse" };

string ilk = liste[0];           // → "Ali"
liste.Insert(1, "Fatma");        // → [Ali, Fatma, Veli, Ayse]
liste.RemoveAt(0);              // → [Fatma, Veli, Ayse]
int sirasira = liste.IndexOf("Veli"); // → 1
```

Bu tür, elemanların sırasının anlamlı olduğu ve konuma göre erişimin gerekli olduğu durumlarda tercih edilir.

8. `List<T>`

Önceki bölümlerde ele alınan türlerin tümü birer arayüz, yani birer sözleşmedir. `List<T>` ise bu sözleşmeleri uygulayan somut bir sınıf olup, `new` işleci ile doğrudan örneği oluşturulabilen tek türdür; arayüzlerin örneği doğrudan oluşturulamaz. `List<T>`; `IList<T>`, `ICollection<T>` ve `IEnumerable<T>` başta olmak üzere çok sayıda arayüzü uygular. İç yapısında dinamik boyutlu bir dizi tutar ve kapasitesi dolduğunda otomatik olarak büyür. Sunduğu üyeler, önceki türlerin tüm yeteneklerini kapsamanın yanı sıra `Sort()`, `AddRange()` ve `Find()` gibi ek işlevleri de içerir.

```
List<int> sayilar = new List<int> { 3, 1, 2 };
sayilar.Add(5);
sayilar.Sort();           // → [1, 2, 3, 5]
sayilar.AddRange(new[] { 8, 13 });
int ikinci = sayilar[1];  // → 2
```

Bir koleksiyonun yöntem içinde oluşturulup üzerinde çeşitli işlemlerin yapılacağı durumlarda `List<T>` uygun bir seçimdir. Bununla birlikte, bu türün yöntem imzasında (parametre ya da dönüş türü olarak) kullanılması çoğunlukla önerilmez; bunun gerekçesi 10. bölümde ele alınmaktadır.

9. İlgili Diğer Türler

Yukarıda ele alınan temel türlerin yanı sıra, belirli senaryolar için tasarlanmış başka koleksiyon türleri de bulunmaktadır. Sabit boyutlu diziler (`T[]`) en yüksek erişim başarımını sunar, ancak boyutları sonradan değiştirilemez. Salt okunur arayüzler olan `ICollection<T>` ve `IReadOnlyList<T>`, bir koleksiyonun sayılabilir veya dolaşılabilir olduğunu ancak değiştirilemeyeceğini ifade etmenin açık bir yolunu sağlar ve genel (public) API'lerde koleksiyon döndürürken tercih edilir. `HashSet<T>`, tekrarsız elemanlardan oluşan bir küme sağlar ve üyelik denetimini yüksek başarımla gerçekleştirir. `Dictionary<TKey, TValue>` ise anahtar-değer eşlemesi sunarak hızlı arama gerektiren durumlar için uygun bir yapı oluşturur.

10. Tür Seçimine İlişkin Uygulama Önerileri

Tür seçimi, kod kalitesini ve sürdürülebilirliği doğrudan etkileyen bir tasarım kararıdır. Bu bağlamda iki temel ilke öne çıkmaktadır.

Parametrelerde mümkün olan en genel tür istenmelidir. Bir yöntem koleksiyonu yalnızca dolaşacaksa `IEnumerable<T>` parametresi tanımlamak yeterlidir; bu sayede çağırın kod, ister `List`, ister dizi, ister `HashSet` olsun, herhangi bir uyumlu koleksiyonu iletebilir.

```
public decimal ToplamHesapla(IEnumerable<Urun> urunler) { ... }
```

Dönüş türünde, çağırının gereksinim duyacağı ancak fazlasını taahhüt etmeyen tür seçilmelidir. Bu doğrultuda; yalnızca okunup dolaşılacak bir sonuç için `IEnumerable<T>` veya `IReadOnlyList<T>`, ekleme ve çıkarma yapılacak bir koleksiyon için `ICollection<T>` veya `IList<T>`, dışarıda daha da filtrelenebilmesi istenen bir Entity Framework Core sorgusu için ise `IQueryable<T>` uygun tercihlerdir.

Not. Bir yöntemin içinde `List<T>` kullanılması olağandır. Ancak sonucun dışarıya `List<T>` olarak döndürülmesi, çağırın kodun bu liste üzerinde değişiklik yapabilmesine olanak tanır ve dönüş türünün ileride değiştirilmesini güçleştirir. Bu tür durumlarda `IReadOnlyList<T>` döndürmek çoğunlukla daha güvenli bir yaklaşımdır.

Aşağıdaki tabloda incelenen türler, temel nitelikleri bakımından özetlenmektedir.

Tablo 1. C#/.NET koleksiyon türlerinin karşılaştırmalı özeti.

Tür	Temel İşlev	Çalışma Yeri	Değiştirilebilir	İndeks Erişimi
<code>IEnumerable<T></code>	Dolaşma (foreach)	Bellek	Hayır	Hayır
<code>IQueryable<T></code>	Sorgu (SQL'e çevrilir)	Veritabanı	Hayır	Hayır
<code>ICollection<T></code>	Sayma, ekleme, çıkarma	Bellek	Evet	Hayır
<code>ICollection<T></code>	İndeksli erişim	Bellek	Evet	Evet
<code>List<T></code>	Tüm işlemler (somut sınıf)	Bellek	Evet	Evet

11. Sonuç

Bu çalışmada C#/.NET platformundaki başlıca koleksiyon türleri, ortak kalıtım hiyerarşisi temelinde karşılaştırmalı olarak incelenmiştir. İnceleme, tür seçiminde belirleyici olan ilkenin, ihtiyaç duyulan en düşük yeteneği sunan türün tercih edilmesi olduğunu ortaya koymaktadır: parametrelerde en genel tür istenmeli, dönüş türünde ise gereğinden fazla taahhütte bulunulmamalıdır.

Entity Framework Core kullanılan uygulamalarda `IEnumerable` ile `IQueryable` arasındaki ayrım özel bir öneme sahiptir. Bu ayrım, sorgunun bellekte mi yoksa veritabanında mı yürütüleceğini belirlediğinden, veri kaynağından tek bir satırın mı yoksa gereksiz yere tüm tablonun mu çekileceği doğrudan bu tercihe bağlıdır. Dolayısıyla koleksiyon türlerinin bilinçli seçimi, yalnızca bir üslup tercihi değil, uygulamanın başarımını ve sürdürülebilirliğini etkileyen bir mühendislik kararıdır.

Kaynaklar

1. Microsoft, "System.Collections.Generic Ad Alanı," *.NET API Belgeleri*, Microsoft Learn. [Çevrimiçi]. Erişim: learn.microsoft.com.
2. Microsoft, "Language Integrated Query (LINQ)," *C# Kılavuzu*, Microsoft Learn. [Çevrimiçi]. Erişim: learn.microsoft.com.
3. Microsoft, "Querying Data — Entity Framework Core," *EF Core Belgeleri*, Microsoft Learn. [Çevrimiçi]. Erişim: learn.microsoft.com.
4. Microsoft, "IQueryable<T> Interface," *.NET API Belgeleri*, Microsoft Learn. [Çevrimiçi]. Erişim: learn.microsoft.com.